

PRACTICAL_WEEK-06

Task 1: Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```
Last login: Fri Sep 18 11:57:24 on ttys030
amrutha@Amruthas-MacBook-Air ~ % cd ~/OSSP
amrutha@Amruthas-MacBook-Air OSSP % pwd
/Users/amrutha/OSSP
amrutha@Amruthas-MacBook-Air OSSP % ls
assets                LICENSE                src
bin                   Makefile              tests
docs                  obj                   wait_flowchart.txt
double_quote_flowchart.txt  parser_flowchart.txt  week3_task3_flowchart.txt
escape_flowchart.txt      process_flowchart.txt  week3_task4_flowchart.txt
File1.txt               quote_flowchart.txt    week5_task1_flowchart.txt
File2.txt               README.md              week5_task2_flowchart.txt
include                scripts
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 1: Navigating to the OSSP project directory before implementing Practical 6 Task

```
UW PICO 5.09

    snprintf(response, sizeof(response),
               "Server processed message: %s", message);

    close(read_fd);

    int write_fd = open(server_to_client, O_WRONLY);
    write(write_fd, response, strlen(response) + 1);
    printf("Response sent to client.\n");

    close(write_fd);

    unlink(client_to_server);
    unlink(server_to_client);

    return 0;
}
```

Fig 2: C program for the FIFO server in the client-server communication system

```

UW PICO 5.09

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>

int main()
{
    char *client_to_server = "/tmp/client_to_server_fifo";
    char *server_to_client = "/tmp/server_to_client_fifo";

    char message[100];
    char response[100];

    printf("Enter message for server: ");
    fgets(message, sizeof(message), stdin);

    message[strcspn(message, "\n")] = '\0';

    int write_fd = open(client_to_server, O_WRONLY);

    write(write_fd, message, strlen(message) + 1);

    close(write_fd);

    int read_fd = open(server_to_client, O_RDONLY);

    read(read_fd, response, sizeof(response));

    printf("Response from server: %s\n", response);

    close(read_fd);

    return 0;
}

```

Fig 3: C program for the FIFO client that sends a message and receives the server response

```

amrutha@Amruthas-MacBook-Air OSSP % cd ~/OSSP
gcc src/week6_server.c -o bin/week6_server
gcc src/week6_client.c -o bin/week6_client

```

Fig 4: Compilation of the FIFO server and client programs

```

amrutha@Amruthas-MacBook-Air OSSP % ./bin/week6_server
Server started...
Waiting for client message...

```

Fig 5: Server process started and waiting for a client message through the named pipe

```

Last login: Sat Sep 19 14:32:35 on ttys004
amrutha@Amruthas-MacBook-Air ~ % cd ~/OSSP
[./bin/week6_client
Enter message for server: Hello Server
Response from server: Server processed message: Hello Server
amrutha@Amruthas-MacBook-Air OSSP % █

```

Fig 6: Client sending a message to the server and receiving the processed response through FIFO

```

Message received from client: Hello Server
Response sent to client.
amrutha@Amruthas-MacBook-Air OSSP % █

```

Fig 7: Server receiving the client message and sending a response through the named pipe

```

amrutha@Amruthas-MacBook-Air OSSP % gcc src/week6_server.c -o bin/week6_server
amrutha@Amruthas-MacBook-Air OSSP % ./bin/week6_server
Server started...
Waiting for client messages...
Client 1 message: ????(r?
Response sent to Client 1

Client 2 message: Message from Client 2
Response sent to Client 2

Client 3 message: Message from Client 3
Response sent to Client 3

Server completed communication with 3 clients.
amrutha@Amruthas-MacBook-Air OSSP % █

```

Fig 8: FIFO server processing messages from multiple client processes

```

amrutha@Amruthas-MacBook-Air OSSP % nano week6_task1_flowchart.txt
amrutha@Amruthas-MacBook-Air OSSP % cat week6_task1_flowchart.txt
+-----+
|      START      |
+-----+
|      v          |
+-----+
| Create Named Pipes (FIFOs) |
| using mkfifo()  |
+-----+
|      v          |
+-----+
| Start Server Process      |
+-----+
|      v          |
+-----+
| Server waits for Client |
| Message               |
+-----+
|      v          |
+-----+
| Client opens FIFO and sends |
| message using write()      |
+-----+

```

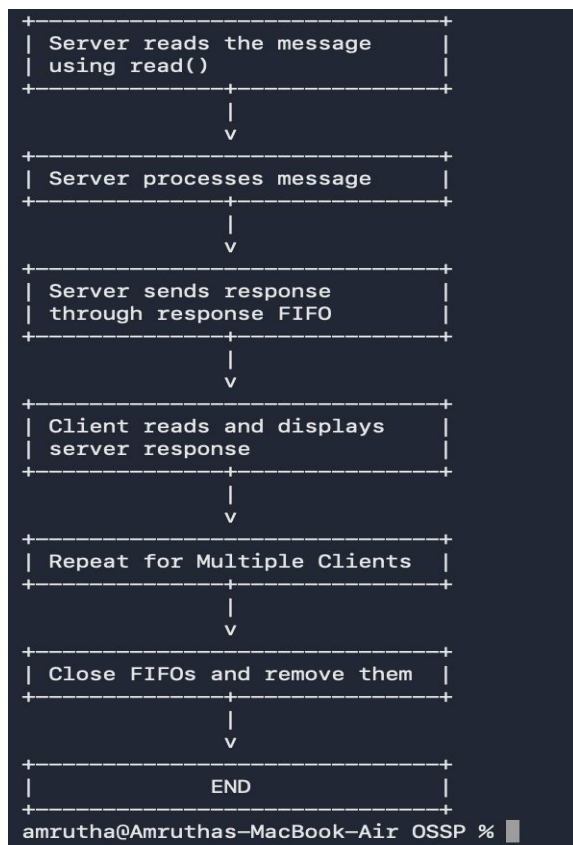


Fig 9: Flowchart of client-server communication using Named Pipes (FIFOs)

Observation:

The client successfully sent messages through the FIFO, and the server received, processed, and responded to them correctly. Multiple client requests were handled sequentially using named pipes.

Result:

The client-server application using Named Pipes (FIFOs) was implemented successfully. The client processes sent messages to the server, and the server received, processed, and returned responses through the FIFO channels. Communication with multiple clients was also demonstrated successfully.

Task 2: Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
UW PICO 5.09

sa.sa_handler = signal_handler;
sigemptyset(&sa.sa_mask);
sa.sa_flags = 0;

sigaction(SIGINT, &sa, NULL);
sigaction(SIGTERM, &sa, NULL);
sigaction(SIGUSR1, &sa, NULL);

printf("Process PID: %d\n", getpid());
printf("Waiting for signals...\n");

while (1)
{
    pause();
}

return 0;
}
```

Fig 10: POSIX signal handling program using sigaction() for SIGINT, SIGTERM, and SIGUSR1

```
amrutha@Amruthas-MacBook-Air OSSP % cd ~/OSSP
[gcc src/week6_task2.c -o bin/week6_task2
amrutha@Amruthas-MacBook-Air OSSP % ./bin/week6_task2
Process PID: 6465
Waiting for signals...
█
```

Fig 11: Execution of the POSIX signal handling program waiting for incoming signals

```
Process PID: 6465
Waiting for signals...
^C
SIGINT received (Ctrl+C)
█
```

Fig 12: Handling the SIGINT signal generated using Ctrl+C

```
amrutha@Amruthas-MacBook-Air ~ % kill -USR1 6465
amrutha@Amruthas-MacBook-Air ~ %
```

```
SIGUSR1 received (User-defined signal)
```

Fig 13: Handling the SIGUSR1 user-defined signal using the kill command

```
amrutha@Amruthas-MacBook-Air ~ % kill -TERM 6465
amrutha@Amruthas-MacBook-Air ~ %
```

```
SIGTERM received. Terminating gracefully...
amrutha@Amruthas-MacBook-Air OSSP %
```

Fig 14: Graceful termination of the process after receiving the SIGTERM signal

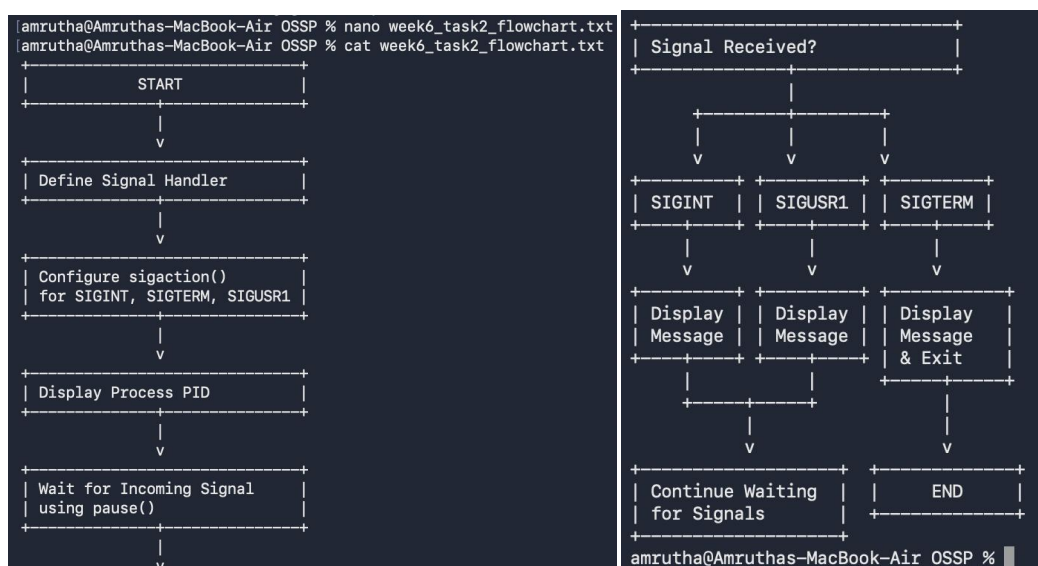


Fig 15: Flowchart of POSIX signal handling for SIGINT, SIGUSR1, and SIGTERM

Observation:

The program successfully handled SIGINT, SIGUSR1, and SIGTERM asynchronously using `sigaction()`. SIGINT and SIGUSR1 displayed their respective messages, while SIGTERM terminated the process gracefully.

Result:

The POSIX signal handling program was implemented successfully. The process captured SIGINT, SIGUSR1, and SIGTERM signals and executed the corresponding signal handler actions correctly.